УЛУЧШЕНИЕ ПРОЕКТА

Реальная техническая дорожная карта развития EH/Unwind Analyzer

Документ объясняет, какие улучшения действительно можно сделать в проекте, зачем они нужны, как они работают, в какие модули встраиваются и в каком порядке их безопасно внедрять. Это не список фантазий: каждое улучшение привязано к текущей архитектуре проекта, к существующим отчетам, динамическому слою, Ghidra-интеграции и тестовой базе.

Критерий	Смысл в этом документе
Важность	Насколько улучшение закрывает реальный технический пробел или повышает достоверность анализа.
Крутость	Насколько заметно улучшение выглядит для заказчика, комиссии или пользователя-реверсера.
Сложность	Реальная трудоемкость с учетом текущего кода на C17, JSON/HTML отчетов, Windows dynamic collector и тестов.
Безопасность внедрения	Насколько можно добавить функцию рядом с текущим ядром без риска сломать проверенную базовую логику.

Версия документа: 2026-08-10. База оценки: текущий проект EH/Unwind Analyzer 1.0.0, модульная C17-архитектура, статический анализ PE32+ AMD64, Windows-only динамика, JSON/HTML/CSV/Ghidra-CSV отчеты, тесты и release-output.

1. Короткий вывод

Проект уже является системой анализа, а не просто парсером: он читает PE/x64 unwind metadata, проверяет структурные нарушения, строит эвристическую оценку, формирует отчеты и умеет собирать часть runtime-доказательств на Windows. Поэтому улучшения должны не переписывать ядро заново, а усиливать его по трем направлениям: глубина анализа, доказательность отчета и удобство использования.

Самое сильное техническое улучшение - добавить сверку UNWIND_CODE с реальными инструкциями пролога функции в секции .text. Сейчас проект хорошо понимает, что записано в .pdata/.xdata, но не доказывает полностью, что эти unwind-операции соответствуют фактическому машинному коду. Это нормальная граница текущей версии, но именно ее логично закрывать первой, если нужно сделать проект еще сильнее.

Второй важный блок - усилить динамику: не только фиксировать, что в процессе были VEH/VCH/UEF, исключения, handler hits и вызовы RtlAddFunctionTable, но и снимать реальное содержимое runtime function tables, которые регистрируются во время исполнения. Это позволит сравнивать статические и runtime-таблицы более предметно.

Третий блок - улучшить отчеты и контракт данных. Для будущей БД, фронтенда и заказчика важен не только факт найденной аномалии, но и объяснимая evidence-модель: где найдено, почему подозрительно, чем подтверждено, чем ослаблено, что проверить вручную, какие ограничения у вывода.

2. Карта приоритетов

ID	Улучшение	Важность	Крутость	Сложность	Что дает
A1	Сверка UNWIND_CODE с реальным прологом .text	P0	Очень высокая	Высокая	Закрывает главный методический вопрос: совпадает ли metadata с кодом функции.
A2	Снимок runtime function tables в динамике	P0	Очень высокая	Средняя/высокая	Позволяет видеть JIT/runtime-таблицы не только как факт вызова API, а как реальные записи.
A3	Парсер __C_specific_handler scope table	P1	Высокая	Средняя	Углубляет работу с language-specific handler data без ложного универсального парсера.
A4	Расширенная классификация handler target	P1	Высокая	Средняя	Лучше объясняет, куда ведет handler RVA: код, thunk, импорт, неисполняемая секция, мусор.
B1	Более богатый JSON schema для БД и фронта	P1	Средняя/высокая	Средняя	Упрощает интеграцию без вмешательства в ядро и делает отчеты стабильным API.
B2	Усиленный HTML-отчет	P1	Высокая	Средняя	Дает заказчику читаемый интерактивный результат, а не только JSON.
B3	Калибровочный corpus benign/anomalous	P1	Средняя	Средняя	Дает аргументы по порогам, ложным срабатываниям и повторяемости.
C1	Ghidra-интеграция с bookmarks по findings	P2	Средняя/высокая	Средняя	Удобнее для реверсера: аномалии сразу видны в дизассемблере.
C2	Профили запуска CLI	P2	Средняя	Низкая	Уменьшает ошибки пользователя: quick/full/static/dynamic-safe режимы.

ID	Улучшение	Важность	Крутость	Сложность	Что дает
C3	DLL/PYD dynamic harness	P2	Средняя	Средняя/высокая	Позволяет динамически проверять библиотеки через контролируемый loader.
C4	CI/release validation gate	P2	Средняя	Средняя	Автоматически не дает собрать релиз, если тесты/схемы/артефакты не сходятся.
D1	Частичный симулятор RtlVirtualUnwind	P3	Высокая	Высокая	Диагностический режим: показать unwind-путь для выбранного адреса.
D2	Экспериментальный C++ EH parser	P3	Высокая	Высокая	Полезно, но рискованно: только как optional-модуль после стабилизации базы.

P0 - самое важное для технической глубины. P1 - сильные улучшения продукта и доказательности. P2 - развитие удобства, инфраструктуры и покрытия. P3 - сложные исследовательские функции, которые не стоит делать первыми.

3. Правило безопасной доработки

Главный принцип: не переписывать проверенный парсер PE/.pdata/.xdata, а добавлять новые анализаторы поверх уже полученной модели. Текущее ядро уже умеет безопасно читать файл, мапить RVA в file offset, декодировать UNWIND_INFO, строить findings и писать отчеты. Если начать менять эти низкоуровневые функции без необходимости, риск регрессий будет выше пользы.

1. Сначала добавить новые структуры результата в include/*.h, не меняя смысл старых полей.
2. Затем добавить отдельный модуль анализа: например prolog_validation.c или handler_data.c.
3. После этого подключить findings в heuristics.c через новые коды и документированные веса.
4. Потом расширить report.c и docs/report.schema.json, сохранив schema_version и тесты совместимости.
5. На каждом шаге прогонять unit tests, regression samples, JSON schema validation и хотя бы один end-to-end отчет.
6. Для динамики проверять Windows smoke отдельно, потому что этот слой выполняет target и зависит от ОС.

Иными словами, улучшения должны входить как слои: parser -> semantic validators -> heuristics -> reports -> tests. Такой порядок дает шанс усилить проект и не сломать уже работающую основу.

A1. Сверка UNWIND_CODE с реальным прологом .text

Параметр	Оценка
Важность	P0 - очень высокая
Крутость	Очень высокая
Сложность	Высокая, но реалистичная

Зачем это нужно

Это главное развитие ядра. Сейчас проект читает unwind metadata и проверяет ее внутреннюю структуру. Но x64 unwind metadata описывает действия пролога функции: сохранение nonvolatile-регистров, изменение RSP, установку frame register, large alloc и другие операции. Поэтому сильная версия анализатора должна уметь хотя бы частично сравнить описание из .xdata с фактическим машинным кодом в .text.

Как это будет работать

Новый слой берет RUNTIME_FUNCTION.BeginAddress, SizeOfProlog и массив UNWIND_CODE. Затем читает первые SizeOfProlog байт функции, выполняет ограниченное дизассемблирование только прологовых инструкций и строит абстрактную модель эффектов: насколько изменился RSP, какие регистры сохранены, был ли frame register, где расположены сохранения. После этого модель сравнивается с тем, что обещает UNWIND_INFO.

Что именно сделать

1. Создать модуль src/prolog_validation.c и include/prolog_validation.h.
2. На первом этапе поддержать минимальный надежный набор x64 пролога: push r64, sub rsp, imm8/imm32, mov rbp/rsp, lea rbp,[rsp+imm], mov [rsp+off], reg, mov [rsp+off], xmm, nop/alignment prefixes.
3. Добавить в модель результат PrologValidation: status, decoded_bytes, unsupported_instruction_count, stack_delta_code, stack_delta_unwind, saved_registers_code, saved_registers_unwind, notes.
4. Добавить findings: prolog_stack_mismatch, prolog_saved_register_mismatch, prolog_frame_register_mismatch, prolog_decode_incomplete.
5. В heuristics.c давать высокий вес только уверенным расхождениям, а unsupported instruction считать не обвинением, а ограничением анализа.
6. В report.c вывести отдельный блок evidence: что сказано в UNWIND_INFO, что найдено в .text, где именно mismatch.
7. Сделать тестовые бинарники: корректный пролог, неверный stack alloc, неверное сохранение RBX, неверный frame register, сложный неподдержанный пролог.

Как проверять

Unit tests на decoder прологовых инструкций; regression PE samples; проверка JSON schema; ручная сверка нескольких функций в Ghidra/llvm-objdump. Важно: если дизассемблер не уверен, он не должен создавать ложный высокий score.

Практический результат

Проект сможет отвечать на вопрос комиссии: metadata не только синтаксически корректна, но и проверяется на соответствие реальному коду функции. Это превращает анализ из структурного в более семантический.

A2. Снимок runtime function tables в динамике

Параметр	Оценка
Важность	P0 - очень высокая
Крутость	Очень высокая
Сложность	Средняя/высокая

Зачем это нужно

Динамический слой уже полезен: он видит исключения, handler hits, VEH/VCH/UEF и вызовы RtlAddFunctionTable/RtlInstallFunctionTableCallback. Но для JIT и runtime-generated code важно не только знать, что таблица зарегистрирована, а увидеть сами runtime RUNTIME_FUNCTION entries.

Как это будет работать

Когда dynamic_win.c ловит RtlAddFunctionTable, он уже получает аргументы API: адрес таблицы, количество entries и base address. Следующий шаг - безопасно прочитать память процесса через ReadProcessMemory, скопировать массив RUNTIME_FUNCTION и прогнать его через тот же валидатор диапазонов, но в runtime-контексте.

Что именно сделать

1. Расширить dynamic event model: добавить runtime_table_snapshot_count, entries_captured, entries_failed, base_address, table_address.
2. В момент API hit читать не больше заранее заданного лимита, например 4096 записей, чтобы target не мог заставить анализатор читать гигантскую память.
3. Добавить DynamicRuntimeFunctionSummary: begin_rva, end_rva, unwind_info_rva, readable, mapped_module, validation_status.
4. Сопоставлять runtime entries со статическими: exact match, new_runtime_only, overlaps_static, points_to_unmapped_memory.
5. В отчет добавить таблицу Runtime Function Tables и связь с итоговым assessment.
6. Сделать тестовый JIT-like sample, который регистрирует корректную таблицу, и мутированный sample с неверным range/unwind pointer.

Как проверять

Windows smoke test с RtlAddFunctionTable; dynamic corpus validation; schema validation; проверка timeout и лимитов чтения памяти.

Практический результат

Динамика перестает быть только подтверждением факта runtime-поведения и начинает давать содержательные данные для сравнения со статикой.

A3. Парсер language-specific данных для __C_specific_handler

Параметр	Оценка
Важность	P1 - высокая
Крутость	Высокая
Сложность	Средняя

Зачем это нужно

После handler RVA в UNWIND_INFO может идти language-specific handler data. Универсально парсить эти данные нельзя: формат зависит от handler. Но для __C_specific_handler можно добавить осознанный parser scope table. Это даст глубокую, но контролируемую поддержку C SEH.

Как это будет работать

Проект сначала определяет handler RVA и пытается понять, что это именно __C_specific_handler: по импорту, имени символа, thunk или known runtime target. Только после такой идентификации он читает handler data как scope table. Если handler неизвестен, проект не притворяется, что понимает формат, а честно пишет unknown handler data.

Что именно сделать

1. Добавить HandlerKind: unknown, c_specific_handler, cpp_handler_candidate, custom_handler.
2. Для c_specific_handler читать Count и записи scope table: BeginAddress, EndAddress, HandlerAddress, JumpTarget.
3. Проверять, что диапазоны scope лежат внутри функции или разумно связаны с ней.
4. Проверять, что handler/filter/jump target указывают в исполняемый код или допустимые sentinel-значения.
5. Добавить findings: scope_table_truncated, scope_range_oob, scope_handler_nonexec, scope_jump_target_oob.
6. Отдельно документировать, что C++ EH и custom runtime не разбираются этим parser-ом.

Как проверять

Минимальные MSVC/MinGW samples с __try/__except; мутированные scope records; отчет должен отличать parse failure от unsupported format.

Практический результат

Проект аккуратно углубляет работу с handler data и не делает методологическую ошибку: не объявляет все language-specific данные универсальным форматом.

A4. Расширенная классификация handler target

Параметр	Оценка
Важность	P1 - высокая
Крутость	Высокая
Сложность	Средняя

Зачем это нужно

Сам по себе handler RVA еще не объясняет, что находится по этому адресу. Для реверсера важно понимать: это настоящая функция, thunk, импортный переход, код в подозрительной секции, неисполняемые данные или вообще unmapped RVA.

Как это будет работать

Классификатор берет handler RVA, мапит его в секцию, проверяет execute/read permissions, смотрит первые байты и связывает это с import/IAT/thunk pattern. Он не обязан полноценно дизассемблировать функцию, но может уверенно отличить несколько типовых классов.

Что именно сделать

1. Добавить HandlerTargetSummary: rva, section, permissions, target_kind, first_bytes, confidence.
2. Распознавать простые thunk patterns: jmp [rip+disp32], jmp rel32, mov rax, imm64; jmp rax.
3. Связывать target_kind с finding severity: nonexec/unmapped - сильный сигнал, thunk/import-like - обычно слабее.
4. Показывать классификацию в JSON, HTML и Ghidra comments.
5. Добавить тесты на handler в .text, .rdata, .pdata, overlay/unmapped, thunk-like bytes.

Как проверять

Unit tests на классификатор байтов; regression samples; проверка, что benign import thunks не получают высокий score без других признаков.

Практический результат

Отчет становится объяснимее: пользователь видит не только адрес handler-a, но и технический смысл этого адреса.

B1. Более богатый JSON schema для БД и фронта

Параметр	Оценка
Важность	P1 - высокая
Крутость	Средняя/высокая
Сложность	Средняя

Зачем это нужно

JSON уже есть, но для будущей базы данных и фронта полезно сделать его еще более удобным как стабильный контракт. Бэкенд сможет брать report.json и сохранять сущности: file, sections, functions, findings, dynamic events, evidence, limitations.

Как это будет работать

Вместо плоской логики только для вывода добавить явные идентификаторы объектов и связи между ними. Например finding ссылается на function_id, rva, section_id, source_layer и evidence_id. Тогда фронт сможет показывать карточку функции, переходы к аномалиям, фильтрацию по severity и сравнение отчетов между файлами.

Что именно сделать

1. Поднять schema_version, например 1.2.0, без ломки старого 1.1.0 либо с migration note.
2. Добавить stable ids: section_index, function_index, finding_id, dynamic_event_id.
3. Добавить source_layer: pe, pdata, xdata, handler, dynamic, report_semantics.
4. Добавить limitation objects: unsupported_arch, unsupported_handler_data, prolog_decode_incomplete, dynamic_not_available.
5. Добавить docs/report.schema.json tests и examples/sample_cmds.txt examples.
6. Сделать короткий документ API_CONTRACT_RU.md для коллеги, который будет делать БД/фронт.

Как проверять

JSON schema validation; semantic validation; snapshot tests для demo_ru.json; тест на backward compatibility или явное сообщение о новой версии схемы.

Практический результат

Коллега сможет строить БД и фронт вокруг ядра без изменения C-кода: ядро остается producer-ом отчетов, backend становится consumer-ом JSON.

B2. Усиленный HTML-отчет

Параметр	Оценка
Важность	P1 - высокая
Крутость	Высокая
Сложность	Средняя

Зачем это нужно

Для реверсера JSON нормален, но для защиты, заказчика и демонстрации HTML важнее: он показывает смысл результатов быстро и наглядно. Это не меняет ядро, но резко повышает воспринимаемую готовность продукта.

Как это будет работать

HTML остается self-contained и offline, без внешних скриптов. Но в него добавляются фильтры, summary cards, раскрывающиеся блоки evidence, таблица severity/family, поиск по RVA и блок manual validation steps.

Что именно сделать

1. В report.c улучшить шаблон HTML: summary cards, severity badges, фильтры по finding family и section.
2. Добавить секцию Static vs Dynamic Correlation: что статически подозрительно, что динамика подтвердила или ослабила.
3. Добавить блок Known Boundaries: динамика только Windows, confidence не вероятность, prolog-vs-text если еще не включен.
4. Сделать navigation anchors: overview, PE validation, functions, findings, dynamic, Ghidra notes.
5. Добавить visual regression: проверить, что HTML содержит CSP, нет external URLs, корректно экранирует пути и имена файлов.

Как проверять

Unit tests на HTML escaping; test_report_semantics.py; ручное открытие demo_ru.html; проверка, что большой отчет не становится нечитаемым.

Практический результат

Инструмент выглядит как законченный продукт: один HTML можно открыть и понять состояние файла без чтения сырого JSON.

B3. Калибровочный corpus benign/anomalous

Параметр	Оценка
Важность	P1 - высокая
Крутость	Средняя
Сложность	Средняя

Зачем это нужно

На защите могут спросить, почему веса и признаки выбраны именно так. Технический ответ сильнее, если есть corpus с нормальными и заведомо поврежденными samples, а не только логическое рассуждение.

Как это будет работать

Создается manifest с файлами, ожидаемыми findings и допустимыми verdict levels. Runner запускает анализатор, проверяет JSON schema и считает confusion-style статистику: сколько benign получили score 0, сколько anomalous получили нужный finding, где есть false positive или false negative.

Что именно сделать

1. Расширить tests/run_regression_samples.py или добавить tests/run_corpus_matrix.py.
2. Сделать corpus_manifest.json: sample, category, expected_findings, expected_min_score, notes.
3. Отдельно хранить synthetic broken binaries и real benign corpus.
4. Добавить score_sweep: посмотреть, как меняется verdict при разных весах.
5. Сформировать отчет calibration_summary.json/html для защиты.
6. В docs/evidence_matrix.md указать, какие признаки подтверждены corpus-тестами.

Как проверять

Сам corpus является тестом. Важно не только PASS/FAIL, но и readable explanation: почему sample считается benign/anomalous.

Практический результат

Появляется инженерное обоснование эвристик и порогов. Это не делает эвристику статистической моделью, но делает ее проверяемой.

C1. Ghidra-интеграция с bookmarks по типам findings

Параметр	Оценка
Важность	P2 - средняя
Крутость	Средняя/высокая
Сложность	Средняя

Зачем это нужно

Ghidra-интеграция уже есть через CSV importer. Следующий реальный шаг - сделать так, чтобы реверсер видел не только диапазоны функций и комментарии, а структурированные bookmarks: parse failure, suspicious handler, chain anomaly, dynamic hit.

Как это будет работать

Анализатор выгружает расширенный CSV или JSON для Ghidra. Скрипт import_unwind_csv.py создает categories bookmarks, tags comments and optionally namespaces. Каждая аномалия получает адрес, уровень severity и короткую причину.

Что именно сделать

1. Добавить поле finding_code и severity в ghidra-csv.
2. В import_unwind_csv.py создать bookmark category EHUA: error/warning/info.
3. Не перетирать пользовательские имена функций и существующие комментарии, только добавлять tagged block.
4. Добавить dry-run режим скрипта для проверки CSV без изменения проекта Ghidra.
5. Сделать пример workflow в docs/GHIDRA_WORKFLOW_RU.md.

Как проверять

Unit-level validation CSV columns; ручная проверка на одном sample в Ghidra; тест, что importer не падает на неизвестной колонке.

Практический результат

Проект лучше встраивается в реальную работу реверсера: отчет превращается в навигацию внутри дизассемблера.

C2. Профили запуска CLI

Параметр	Оценка
Важность	P2 - средняя
Крутость	Средняя
Сложность	Низкая

Зачем это нужно

Сейчас у CLI много опций, это гибко, но новичок легко запустит неполный режим. Профили уменьшают человеческие ошибки и помогают демонстрации.

Как это будет работать

Добавить presets: --profile quick, --profile full-static, --profile dynamic-safe, --profile release-evidence. Каждый profile раскрывается в набор текущих опций, но пользователь может переопределить отдельные параметры.

Что именно сделать

1. В main.c добавить parser для --profile.
2. Документировать, какие опции включает каждый profile.
3. Выводить resolved command options в report_metadata.
4. Добавить ошибку, если dynamic profile запущен не на Windows.
5. Добавить tests/test_cli_contract.py cases.

Как проверять

CLI contract tests; проверка help text; snapshot command_line в JSON.

Практический результат

Инструмент легче отдавать другим людям: можно сказать “запусти full-static” вместо длинной команды.

C3. DLL/PYD dynamic harness

Параметр	Оценка
Важность	P2 - средняя
Крутость	Средняя
Сложность	Средняя/высокая

Зачем это нужно

Текущая динамика работает через CreateProcess и не может напрямую запускать DLL/PYD. Это корректное ограничение, но в реальном мире много интересных PE x64 - именно библиотеки.

Как это будет работать

Добавляется маленький controlled loader executable, который загружает DLL через LoadLibraryEx, вызывает безопасную exported entry или просто держит процесс живым под debugger-ом. Analyzer запускает loader, а не саму DLL. В отчете явно пишется, что dynamic target был library harness.

Что именно сделать

1. Создать tools/dll_harness.c или tests/dll_harness.c.
2. Добавить CLI опцию --dynamic-loader-harness path или автоматический режим для DLL/PYD.
3. Не вызывать произвольные exports без явного указания пользователя.
4. Соблюдать timeout, sandbox/VM warning и логировать loader events.
5. Сделать samples: DLL with static unwind only, DLL registering function table in DllMain, DLL requiring dependency failure.

Как проверять

Windows integration tests; negative tests на missing dependency; dynamic report schema validation.

Практический результат

Динамический слой станет полезнее для библиотек, не ломая безопасность базового CreateProcess-подхода.

C4. CI/release validation gate

Параметр	Оценка
Важность	P2 - средняя
Крутость	Средняя
Сложность	Средняя

Зачем это нужно

Для релиза важно не только, что проект работает один раз на машине автора, а что сборка воспроизводима. Сейчас есть release scripts and tests; следующий шаг - сделать жесткий gate, который не выпускает архив при несоответствии.

Как это будет работать

Release script перед упаковкой проверяет версию, schema, CTest, demo reports, hashes, отсутствие build/cache мусора и наличие verified Windows artifact. Если хотя бы один gate не пройден, архив не создается или создается как RC, но не final.

Что именно сделать

1. Расширить scripts/create_release_bundles.py: режим --strict-release.
2. Добавить release_manifest.json с tool_version, schema_version, build_compiler, artifact hash, tests_passed_at.
3. Автоматически генерировать SHA256SUMS и short release note.
4. Добавить test_release_hygiene.py проверку, что docs/ru и samples не потеряны.
5. Для Windows build сохранить отдельный VERIFIED_WINDOWS_BUILD.json.

Как проверять

Release metadata tests; dry-run packaging; проверка, что stale exe не попадает в финальный архив.

Практический результат

Заказчику можно передать папку или архив с понятным доказательством: эта сборка прошла такие-то проверки.

D1. Частичный симулятор RtlVirtualUnwind

Параметр	Оценка
Важность	P3 - после базы
Крутость	Высокая
Сложность	Высокая

Зачем это нужно

Это очень эффектная функция для обучения и диагностики: пользователь задает ControlPc/RVA, а проект показывает, какой RUNTIME_FUNCTION выбран, какой UNWIND_INFO прочитан и как теоретически будет восстановлен caller context. Но это сложнее, чем обычный parser.

Как это будет работать

Режим --simulate-unwind <rva> находит covering runtime function, декодирует unwind ops, моделирует изменения RSP/RBP/RIP на абстрактном stack snapshot и показывает шаги. На первом этапе это не полный эмулятор Windows, а diagnostic trace для понятных случаев.

Что именно сделать

1. Добавить отдельный модуль virtual_unwind_sim.c, не смешивать с parser-ом.
2. Ограничиться unwind info без сложных chained/handler edge cases на первой итерации.
3. Ввести статус: simulated, partial, unsupported, invalid.
4. Добавить подробный textual trace в HTML/JSON.
5. Сделать несколько эталонных tests на простые функции.

Как проверять

Unit tests на простые UWOP_PUSH_NONVOL, UWOP_ALLOC_SMALL, UWOP_SET_FPREG; comparison с Windows RtlVirtualUnwind для нескольких generated samples.

Практический результат

Функция сильная для демонстрации, но ее стоит делать после A1/A2, иначе проект уйдет в сложную эмуляцию раньше, чем закроет основную проверку metadata-vs-code.

D2. Экспериментальный C++ EH parser

Параметр	Оценка
Важность	P3 - осторожно
Крутость	Высокая
Сложность	Высокая

Зачем это нужно

C++ exception handling metadata очень заманчиво разбирать, но это рискованная область: форматы зависят от MSVC runtime, версии компилятора и ABI-деталей. Если сделать плохо, проект начнет выдавать уверенные ошибки там, где просто не понял runtime.

Как это будет работать

Делать только как optional experimental parser. Сначала определить candidate handler kind, затем читать только документированные/известные поля, а при малейшей неуверенности возвращать unsupported вместо anomaly. Основная эвристика не должна сильно зависеть от экспериментального parser-а.

Что именно сделать

1. Добавить compile-time или CLI flag --experimental-cpp-eh.
2. Собрать маленький corpus MSVC C++ try/catch samples разных вариантов.
3. Ввести поле experimental=true в JSON для результатов этого parser-а.
4. Не включать высокий suspicion score только из-за непонятного C++ EH формата.
5. Документировать ограничения в docs/limitations или HTML report.

Как проверять

Только после corpus. Нужны positive и negative samples, иначе риск ложной уверенности слишком велик.

Практический результат

Может стать красивой исследовательской веткой, но не является первым улучшением для финального продукта.

D3. Небольшой backend adapter без вмешательства в ядро

Параметр	Оценка
Важность	P2 - прикладная
Крутость	Средняя/высокая
Сложность	Средняя

Зачем это нужно

Если коллега делает БД и фронт, ядро менять не обязательно. Но удобно дать тонкий adapter, который запускает CLI, принимает путь к бинарнику, сохраняет JSON/HTML и возвращает report id. Это не микросервисная перестройка ядра, а оболочка вокруг готового CLI.

Как это будет работать

Backend вызывает eh_unwind_analyzer как subprocess с заранее заданным profile, получает report.json, валидирует schema, сохраняет в БД и отдает фронту. Ядро остается независимым консольным инструментом, а adapter отвечает за upload, job queue, storage и access control.

Что именно сделать

1. Сделать минимальный docs/API_ADAPTER_CONTRACT_RU.md: вход, выход, exit codes, где лежат отчеты.

2. Добавить пример wrapper script: run_analysis_job.py, который не меняет C-код.
3. Фиксировать рабочую директорию, temp output и cleanup policy.
4. Для untrusted binaries запретить dynamic mode по умолчанию или требовать VM worker.
5. Сохранить SHA-256 input в БД как главный ключ сравнения отчетов.

Как проверять

Integration test: wrapper запускает CLI на sample, проверяет exit code, JSON schema, наличие HTML.

Практический результат

Проект легче встраивать во фронт/БД, но архитектура ядра остается простой и устойчивой.

4. Рекомендуемый порядок работ

Если развивать проект после текущей версии, лучше не хвататься за все сразу. Ниже порядок, который дает максимальный эффект и минимальный риск.

Этап	Содержание	Почему именно так
1	B2 HTML report + B1 JSON schema polishing + C2 CLI profiles	Быстро усиливает продуктовую готовность, почти не трогает низкоуровневый parser. Хорошо для заказчика и фронтенда.
2	B3 corpus calibration + release gate C4	Дает доказательность. Перед углублением анализа нужно иметь надежный способ ловить регрессии.
3	A2 runtime function table snapshots	Развивает динамику через уже существующие hooks и дает сильную static/dynamic correlation.
4	A1 prolog-vs-unwind validation	Главное, но сложное улучшение. Его надо делать после тестовой базы, чтобы не получить ложные срабатывания.
5	A3 __C_specific_handler + A4 handler target classifier	Углубляет handler layer, когда базовая модель function/unwind/prolog уже крепкая.
6	C1 Ghidra bookmarks + C3 DLL harness	Удобство для реверсера и расширение динамики на библиотеки.
7	D1 simulate unwind + D2 C++ EH experimental	Исследовательский слой. Красиво, но только после стабилизации основной семантики.

5. Что не стоит делать прямо сейчас

- Не переписывать проект на другой язык только ради модности. C17 здесь уместен: бинарные структуры, контроль памяти, кроссплатформенная статическая часть и Windows API для динамики.
- Не делать нейросетевый поиск аномалий как замену эвристикам. Для PE/EH/Unwind важна объяснимость: комиссия и реверсер должны видеть конкретное поле, RVA, секцию и правило.
- Не обещать универсальный parser для всех language-specific handler data. Корректный путь - parser по известному handler kind и honest unsupported для остальных.
- Не делать полный x64 дизассемблер в первом же шаге. Для A1 нужен ограниченный prolog decoder, а не попытка заменить Capstone/Ghidra/IDA.
- Не включать dynamic mode по умолчанию для неизвестных файлов. Он выполняет target и должен использоваться в VM/isolated environment.
- Не ломать schema без версии. Если меняются JSON-поля, нужен schema_version и tests, иначе backend/front потеряют стабильный контракт.

6. Как объяснять улучшения на защите

Правильная формулировка: проект уже закрывает структурный анализ PE32+ AMD64 EH/Unwind metadata, формирует отчеты и собирает часть runtime evidence. Следующий логичный этап - не переписывание, а повышение семантической глубины: связать metadata с реальными инструкциями пролога, расширить runtime snapshot, уточнить handler data для известных handler-ов и сделать evidence удобнее для БД, фронта и Ghidra.

Если спрашивают, какое улучшение самое важное, ответ: сверка UNWIND_CODE с .text. Потому что она проверяет не только внутреннюю корректность .xdata, а соответствие между описанием unwind-поведения и фактическим кодом функции. Это наиболее прямое усиление цели проекта.

Если спрашивают, почему не сразу C++ EH или полный эмулятор RtlVirtualUnwind, ответ: потому что это сложные runtime-specific задачи. Их нужно делать после corpus-калибровки и prolog-vs-unwind проверки, иначе можно получить красивую, но менее надежную систему.

Если спрашивают, что реально можно сделать быстро, ответ: улучшить HTML/JSON контракт, добавить CLI profiles, расширить Ghidra CSV, усилить corpus и release gate. Эти задачи не требуют ломать ядро и дают видимый эффект.

7. Итоговая рекомендация

Для максимального, но реалистичного развития проекта я бы выбрал такой набор работ: сначала усилить отчеты и JSON-контракт, затем закрепить corpus и release gate, после этого добавить runtime table snapshots, и только затем внедрять prolog-vs-unwind validator. Такой порядок аккуратный: он сохраняет текущую работоспособность, повышает доказательность и постепенно добавляет более глубокий анализ.

Самая важная мысль: улучшения должны быть проверяемыми. Каждая новая возможность обязана давать не абстрактное “подозрительно”, а конкретное evidence: какое поле, какой RVA, какая секция, какое правило нарушено, насколько анализ уверен и что пользователь может проверить вручную.

8. Ключевые ограничения текущей версии и направления развития

Этот раздел нужен именно как ответ комиссии. В нем важно не говорить “проект плохой, потому что не делает все на свете”. Правильная позиция другая: текущая версия сознательно закрывает структурный анализ и triage PE x64 EH/Unwind metadata. Более глубокие семантические функции требуют отдельных модулей, потому что они работают уже не только с форматом .pdata/.xdata, а с машинным кодом, контекстом стека, runtime handlers и language-specific структурами.

Главная формулировка: ограничения текущей версии - это не ошибки текущего ядра, а границы выбранного уровня анализа. Проект уже автоматизирует извлечение, проверку, связывание и оценку EH/Unwind metadata. Следующие четыре направления переводят инструмент с уровня структурного анализа на уровень более глубокой семантической реконструкции.

8.1. Четыре главных функции, которых сейчас нет

Блок	Что сейчас не делает проект	Почему это важно	Где описано развитие
1	Не сравнивает UNWIND_CODE с реальным прологом .text.	Без этого проект видит, что metadata структурно корректна, но не доказывает, что она совпадает с инструкциями push/sub/mov/lea в функции.	A1. Сверка UNWIND_CODE с реальным прологом .text
2	Не симулирует RtlVirtualUnwind пошагово.	Без ControlPc, RSP, register context и памяти стека нельзя показать восстановление caller RIP/registers как делает Windows.	D1. Частичный симулятор RtlVirtualUnwind
3	Не реконструирует __C_specific_handler scope table.	Проект видит handler RVA, но не строит цепочку C SEH: scope range, filter, handler target, условный переход при EXECUTE_HANDLER.	A3. Парсер language-specific данных для __C_specific_handler
4	Не парсит универсально все language-specific handler data.	Формат зависит от конкретного handler-а и runtime-а. Универсальный parser дал бы ложные выводы.	A3 и D2, но с ограничением: только known handler kind

8.2. Почему эти блоки нельзя просто “быстро дописать в парсер”

Все четыре направления находятся выше обычного парсинга. Обычный parser отвечает на вопрос: “правильно ли прочитаны поля структуры”. Эти блоки отвечают на более сложные вопросы: “совпадает ли metadata с машинным кодом”, “как Windows восстановит контекст”, “какой runtime handler будет читать эти данные”, “какие language-specific структуры находятся за handler RVA”. Поэтому их нельзя аккуратно добавить одной функцией без архитектурного слоя, тестов и расширения отчетов.

Направление	Почему сложнее парсинга
Prolog vs unwind	Нужно читать байты .text, декодировать subset x64-инструкций, построить модель stack/register effects и сопоставить ее с UWOP.
RtlVirtualUnwind simulation	Нужен ControlPc, register context, stack memory, модель восстановления nonvolatile registers и return address. Для static-only файла этих данных нет.
C SEH scope table	Нужно надежно определить handler kind, найти handler data, разобрать scope records, отличить C SEH от C++ EH/custom runtime.
Language-specific data	Windows знает, что данные есть, но универсально не обязана понимать их формат. Формат задает handler/runtime.

8.3. Блок 1: сверка UNWIND_CODE с реальным прологом .text

Сейчас проект читает UNWIND_INFO и понимает, что metadata говорит: например, функция сохраняет RBX, выделяет 0x40 байт стека, использует frame register. Но он не открывает .text и не проверяет, есть ли в первых SizeOfProlog байтах реальные инструкции push rbx, sub rsp, 40h, mov/lea frame

pointer и т.п.

Почему это важно: атакующий или packer может оставить синтаксически валидную metadata, которая плохо соответствует коду. Или наоборот, необычный compiler output может выглядеть странно, но быть корректным. Сравнение metadata с кодом дает более сильное evidence, чем просто проверка полей .xdata.

1. Добавить модуль prolog_validation.c, который работает поверх уже разобранных RuntimeFunction и UnwindSummary.
2. Читать из .text только диапазон BeginAddress..BeginAddress+SizeOfProlog.
3. Поддержать ограниченный и надежный subset x64 prolog instructions: push r64, sub rsp, imm, mov/lea frame register, saves to [rsp+offset].
4. Построить модель действий кода: stack_delta, saved_registers, frame_register_effect.
5. Построить модель действий metadata из UWOP.
6. Сравнить две модели и выдать status: match, mismatch, partial, unsupported.
7. Добавить findings: prolog_stack_mismatch, prolog_saved_register_mismatch, prolog_frame_register_mismatch, prolog_decode_incomplete.

> Как отвечать комиссии: “В текущей версии это вынесено в развитие, потому что требует отдельного мини-дизассемблера или подключения дизассемблерной библиотеки. В ядро парсинга это нельзя добавлять наспех: иначе возрастет риск ложных срабатываний.”

8.4. Блок 2: пошаговая симуляция RtlVirtualUnwind

Сейчас проект показывает структуру unwind metadata, но не выполняет процедуру вида: ControlPc попал в функцию, найден RUNTIME_FUNCTION, прочитан UNWIND_INFO, текущий RSP такой-то, после undo allocation RSP стал таким-то, RBX восстановлен отсюда, return address прочитан отсюда, caller RIP такой-то.

Почему это важно: такой режим превращает unwind из черного ящика в объяснимую процедуру. Он особенно полезен для crash dump analysis, обучения и проверки того, как Windows доверяет .pdata/.xdata при восстановлении caller context.

Что нужно для симуляции	Почему текущий static report этого не имеет
ControlPc	Нужен конкретный адрес исполнения, а не только файл целиком.
Register context	Нужны RIP/RSP/RBP и другие регистры на момент исключения или анализа.
Stack memory	Чтобы реально восстановить RBX/RBP/return address, нужно читать память стека.
Unwind operation semantics	Нужно не только знать UWOP, но и применить их в правильном порядке к контексту.
Confidence model	Нужно учитывать unsupported op, chained info, handler phase, prolog/epilog position и неполные данные.

Реалистичная реализация: добавить режим --simulate-unwind <control-pc> и принимать контекст из файла/JSON/crash dump/live process. Для первой версии можно сделать partial simulator для простых случаев и честно писать unsupported там, где данных недостаточно.

8.5. Блок 3: реконструкция __C_specific_handler и C SEH scope table

Сейчас проект видит, что UNWIND_INFO имеет EHANDLER/UHANDLER и handler_rva. Он может показать, где handler расположен и подозрительны ли права секции. Но он не определяет надежно, что это именно __C_specific_handler, и не разбирает handler data как C SEH scope table.

Почему это важно: для frame-based C SEH пользователь хочет видеть не просто “есть handler”, а цепочку доказательств: функция, .pdata entry, UNWIND_INFO flags, handler RVA, handler kind, handler data, scope ranges, filter candidate, handler target, interpretation. Такой отчет действительно учит и

показывает путь рассуждения.

Нужно добавить	Смысл
handler_kind	Определить unknown / c_specific_handler / cpp_handler_candidate / custom_handler.
handler_data_rva/fileoff	Сохранить точку, где после handler RVA начинаются language-specific данные.
scope_table parser	Для __C_specific_handler читать count и scope records.
scope validation	Проверять range, filter, handler target, executable placement, truncation.
report evidence chain	Показывать Function -> .pdata -> .xdata -> handler -> handler data -> interpretation.

Главное правило: этот parser должен включаться только когда handler достаточно надежно распознан как __C_specific_handler. Если handler неизвестен, проект должен писать unknown/unsupported, а не насильно трактовать данные как C scope table.

8.6. Блок 4: language-specific handler data в целом

Language-specific handler data - это данные после handler RVA, которые будет читать конкретный language/runtime handler. Windows знает, где они находятся, но не обязана универсально понимать их внутренний формат. Поэтому одна универсальная функция “parse all handler data” была бы неправильной архитектурой.

Handler/runtime	Как относиться
__C_specific_handler	Можно реализовать parser scope table как первый безопасный target.
C++ EH handler	Лучше отдельный experimental parser после corpus и документации, потому что формат сложнее и зависит от MSVC runtime.
Custom handler	Не парсить внутренний формат, показывать handler RVA, handler data location и ограничения.
JIT/custom runtime	Сначала связывать со статикой и dynamic evidence, но не делать уверенную реконструкцию без знания runtime-а.

> Как отвечать комиссии: “Проект не парсит все language-specific данные универсально, потому что это было бы технически неверно. Корректная стратегия - поддерживать известные handler kinds отдельными модулями и явно показывать unsupported для неизвестных runtime-форматов.”

8.7. Влияет ли это на статус текущего проекта

Эти четыре блока не отменяют текущую готовность проекта как инструмента структурного анализа. Они расширяют его уровень. Текущий проект уже полезен как automated structural analyzer и triage tool. Добавление этих блоков превратит его в более глубокий semantic analysis tool.

Вопрос комиссии	Сильный ответ
Почему проект не делает все четыре функции сейчас?	Потому что это отдельные семантические слои: дизассемблирование, симуляция stack unwind, runtime-specific handler parsing. Их нельзя смешивать с базовым parser-ом без риска ложных выводов.
Это значит, что проект просто parser?	Нет. Parser - нижний слой. Проект уже делает validation, findings, score, confidence, dynamic evidence, reports and manual steps.
Это критичный недостаток?	Для цели “структурный анализ и выявление аномалий PE x64 EH/Unwind metadata” - нет. Для цели “полная семантическая реконструкция exception handling” - это развитие следующего этапа.
Почему не добавить универсальный language-specific parser?	Потому что формат данных зависит от handler-а. Универсальный parser дал бы ложную уверенность.
Какой модуль делать первым?	Сначала prolog-vs-unwind validator или __C_specific_handler parser. Они дают самый понятный прирост доказательности.

8.8. Реалистичный план внедрения этих четырех блоков

Этап	Что сделать	Риск	Проверка
1	Добавить handler_data_rva/fileoff в модель отчета без парсинга scope table.	Низкий	Unit tests на offset/padding/tail layout; JSON schema update.
2	Добавить handler_kind classifier: known/unknown/c_specific_handler candidate.	Средний	Samples с MSVC/MinGW handlers, unknown custom handler.
3	Добавить __C_specific_handler scope table parser как optional module.	Средний/высокий	C SEH corpus: нормальные и поврежденные scope tables.
4	Добавить prolog decoder для ограниченного subset x64.	Высокий	Generated samples: push/sub/frame/saves/mismatch/unsupported.
5	Добавить prolog-vs-unwind findings и HTML/JSON evidence chain.	Средний	Report schema validation, false positive review on benign corpus.
6	Добавить partial RtlVirtualUnwind simulator только для простых контекстов.	Высокий	Comparison with Windows RtlVirtualUnwind on controlled samples.
7	Добавить experimental C++ EH parser только после corpus.	Высокий	MSVC versioned corpus, strict unsupported behavior.

8.9. Как это связать с текущими документами

В этом документе блоки уже представлены как roadmap: A1 - сверка UNWIND_CODE с прологом, A3 - parser __C_specific_handler handler data, D1 - partial RtlVirtualUnwind simulator, D2 - experimental C++ EH parser. Новый раздел собирает их в один “ответ комиссии”, чтобы было понятно: текущая версия ограничена структурным анализом, а развитие переводит проект в семантический анализ.

8.10. Итоговая формулировка для защиты

> “В текущей версии проект сознательно реализует надежный структурный анализ PE x64 EH/Unwind metadata: он извлекает .pdata/.xdata, проверяет форматные инварианты, строит findings, score, confidence, отчеты и динамическую корреляцию. Он не претендует на полную реконструкцию всего exception handling runtime. Более глубокие функции - сверка UNWIND_CODE с реальным прологом, симуляция RtlVirtualUnwind, реконструкция __C_specific_handler scope table и parser language-specific data - вынесены как отдельные направления развития, потому что требуют дизассемблирования, контекста стека и знания конкретного handler/runtime формата.”

> “Это не делает текущую версию просто парсером. Parser - это нижний слой. Над ним уже есть validation, эвристики, evidence model, findings, dynamic validation и отчеты. Но дальнейшее развитие действительно может усилить проект до уровня более глубокого semantic reverse engineering assistant.”

Раздел 9. Сценарная модель развития static + dynamic

Этот раздел добавлен к документу "УЛУЧШЕНИЕ ПРОЕКТА" как практическая дорожная карта: какие сценарии уже покрываются текущим ядром, какие покрываются частично, а какие требуют развития отчетного, статического и динамического слоев.

9.1. Зачем нужна сценарная модель

В текущей версии проект умеет разобрать PE x64, пройти от Exception Directory к таблице RUNTIME_FUNCTION, прочитать UNWIND_INFO, зафиксировать структурные нарушения, посчитать эвристики и выдать JSON/HTML отчет. Это хорошая инженерная основа. Но профессиональный инструмент для реверс-инженера должен в идеале мыслить не только отдельными признаками, а сценариями расследования.

Сценарий связывает несколько уровней: где лежит metadata, корректно ли она читается, какой handler заявлен, есть ли chain, как это выглядит в статике и было ли runtime-подтверждение. Такой подход делает отчет не набором предупреждений, а объяснимой цепочкой: PE -> Exception Directory -> RUNTIME_FUNCTION -> UnwindInfoAddress -> UNWIND_INFO field -> anomaly -> interpretation -> next manual/dynamic action.

Сценарий	Текущий статус	Что нужно улучшить
1. Нормальный read-only UNWIND_INFO	Частично покрыт	Добавить сверку UWOP с прологом .text
2. CountOfCodes/slots обрываются	Покрыт хорошо	Добавить expected/available size и byte dump
3. EHANDLER -> __C_specific_handler -> scope table	Покрыт только на уровне handler RVA	Парсер handler data и C scope table
4. Writable/executable handler + dominance + exception trigger	Покрыт частично	Связать static findings с exception route и RIP trace
5. CHAININFO cycle	Цикл ловится частично	Вывести полный chain path
6. Runtime RtlAddFunctionTable/JIT table	Фиксируются API-события	Снимать и парсить runtime function table из памяти

9.2. Сценарий 1: нормальная unwind metadata

Картина. Картина: UNWIND_INFO расположен в обычной read-only metadata-секции, version корректная, flags без handler-а, CountOfCodes корректный, операции внутренне допустимы. Для такой функции static может уверенно записать normal unwind metadata. Dynamic не обязан ничего подтверждать, если нет подозрительных статических признаков и нет задачи проверить конкретный exception path.

Что уже делает текущий проект. Что уже делает проект: определяет секцию для UnwindInfoAddress, показывает права секции, version, flags, CountOfCodes, stack summary, unwind_ok и отсутствие anomalies. В JSON/HTML это уже видно через functions[].

Что нужно развить. Что нужно развить: добавить prolog-vs-unwind validation. Инструмент должен дизассемблировать первые SizeOfProlog байт функции и проверить, что UWOP_PUSH_NONVOL, UWOP_ALLOC_SMALL/LARGE, UWOP_SET_FPREG и save-операции объясняются реальным кодом. Тогда отчет сможет писать не только "UNWIND_INFO parsed", а "metadata agrees with function prolog".

9.3. Сценарий 2: CountOfCodes требует slots, которых нет

Картина. Картина: заголовок UNWIND_INFO читается, но массив UNWIND_CODE неполный. Например, одна операция UWOP_ALLOC_LARGE требует дополнительный slot или два slots, но CountOfCodes заканчивается раньше. Это malformed-состояние. Static должен остановить глубокий разбор этой записи и не строить handler/chained выводы из мусора.

Что уже делает текущий проект. Что уже делает проект: это один из наиболее хорошо закрытых случаев. Парсер считает количество slots для операции, проверяет i + slots <= CountOfCodes и при нарушении возвращает parse failure. Ошибка попадает как unwind_parse_fail/unwind_error, а запись не должна использоваться как надежный источник handler-семантики.

Что нужно развить. Что нужно развить: добавить в отчет точные поля evidence: failed_field=CountOfCodes/unwind_code_slots, expected_slots, available_slots, expected_size_bytes, available_size_bytes, file_offset, section, byte dump вокруг проблемного места. Это превратит вывод из "parse failed" в проверяемое доказательство.

9.4. Сценарий 3: EHANDLER, __C_specific_handler и scope table

Картина. Картина: flags содержат EHANDLER, handler RVA указывает на __C_specific_handler, а после handler RVA лежат language-specific данные, похожие на C scope table. Protected ranges находятся внутри функции. Это может быть нормальная C SEH-логика, и хороший static не должен автоматически пугать пользователя.

Что уже делает текущий проект. Что уже делает проект: видит EHANDLER/UHANDLER, извлекает handler_rva, показывает handler_section, может проверять размещение handler-а и reuse/dominance. Этого достаточно, чтобы сказать: у функции есть handler target и он расположен там-то.

Что нужно развить. Что нужно развить: добавить идентификацию известных language/runtime handlers и парсер хотя бы для MSVC C SEH scope table. Отчет должен показывать Count, protected range, filter candidate, handler target, границы внутри функции и confidence. Важно: language-specific data нельзя парсить универсально без знания handler-а, поэтому модуль должен быть handler-aware.

9.5. Сценарий 4: writable/executable handler, dominance и exception-based control flow

Картина. Картина: flags говорят EHANDLER, handler RVA ведет в writable executable section, один неизвестный handler обслуживает сотни функций, а рядом в коде есть deliberate RaiseException или int 3. Это уже сильная связка сигналов: возможно exception-based control flow, protection runtime, обфускация или dispatcher-oriented обработка.

Что уже делает текущий проект. Что уже делает проект: умеет фиксировать handler_writable_section, handler_dominance, reuse, suspicious placement, dynamic exceptions, handler hits и часть runtime events. Это важная основа для подозрения.

Что нужно развить. Что нужно развить: связать эти признаки в один сценарный finding. Статический слой должен искать явные exception triggers в .text, например int 3, ud2, RaiseException/import thunk/call pattern, а динамический слой должен показывать route: exception address -> handler hit -> next RIP samples -> module/section transition. Тогда отчет сможет обосновывать не просто "handler suspicious", а "exception route реально похож на control-flow mechanism".

9.6. Сценарий 5: CHAININFO cycle

Картина. Картина: flags содержат CHAININFO, но chain возвращается к уже посещенному UNWIND_INFO. Это серьезно: анализ primary info может уйти в цикл, а ручной разбор становится опасным, если человек не фиксирует visited set.

Что уже делает текущий проект. Что уже делает проект: следует по chain ограниченно, хранит посещенные RVA и ставит anomaly chain_loop/depth exceeded. То есть сам факт цикла или чрезмерной глубины уже может быть обнаружен.

Что нужно развить. Что нужно развить: выводить полный chain path в отчет. Не только "chain_loop", а последовательность вида 0x4010 -> 0x4050 -> 0x4090 -> 0x4050, глубину, точку повторного входа и связанную RUNTIME_FUNCTION. Это особенно полезно в HTML и JSON, потому что человек сразу видит маршрут ошибки.

9.7. Сценарий 6: disk .xdata нормальная, но runtime регистрирует новую таблицу

Картина. Картина: на диске .pdata/.xdata выглядят нормально, но при запуске target вызывает RtlAddFunctionTable или RtlInstallFunctionTableCallback и регистрирует новую unwind-карту для JIT/payload/memory-generated code. Static видит только часть картины, потому что новая function table появляется во время исполнения.

Что уже делает текущий проект. Что уже делает проект: dynamic-слой может фиксировать вызовы RtlAddFunctionTable/RtlInstallFunctionTableCallback и runtime events. Это дает важный сигнал: unwind map может быть не только статической, но и runtime-регистрируемой.

Что нужно развить. Что нужно развить: читать аргументы RtlAddFunctionTable из процесса, снимать массив RUNTIME_FUNCTION из памяти target, мапить BaseAddress/Begin/End/UnwindInfoAddress, парсить runtime UNWIND_INFO, связывать exceptions внутри этого диапазона с handler routes. В отчете это должно появляться как отдельная runtime function table map.

9.8. Приоритет внедрения

Приоритет 1. Evidence-report без изменения логики парсинга. Добавить failed_field, expected/available sizes, byte dump, section permissions, per-function diagnostic block, полный chain path. Это наиболее безопасное улучшение: оно почти не меняет ядро анализа, но резко повышает доказательность отчета.

Приоритет 2. Улучшение CHAININFO и CountOfCodes диагностики. Хранить route chain traversal и подробные причины остановки парсера. Это локально связано с unwind.c/report.c и хорошо тестируется синтетическими бинарниками.

Приоритет 3. Runtime function table snapshot. В dynamic-слое сохранять не только факт вызова RtlAddFunctionTable, но и содержимое таблицы из памяти target. Это важный шаг к полноценной static/dynamic корреляции.

Приоритет 4. Handler-aware parsing. Добавить распознавание __C_specific_handler и базовый разбор C scope table. Это повышает объяснимость EHANDLER-сценариев, но требует аккуратного ограничения области применимости.

Приоритет 5. Prolog-vs-unwind validation. Добавить дизассемблерный модуль и сверять UWOP с реальным прологом. Это очень ценно, но сложнее и рискованнее, поэтому лучше делать после усиления отчетного слоя.

9.9. Формулировка для защиты

Текущий проект реализует основу двухслойной системы: static-анализ PE/.pdata/.xdata, эвристики и часть dynamic evidence. Он уже способен обнаруживать поврежденные unwind-структуры, подозрительное размещение metadata/handler targets, reuse/dominance и runtime-события. При этом идеальная сценарная модель требует дальнейшего развития: более подробного evidence-report, полной chain path, парсинга handler-specific data, snapshot runtime function tables и сверки unwind metadata с реальным прологом функции. Эти улучшения являются естественным развитием проекта, а не исправлением концептуальной ошибки.

ДОПОЛНЕНИЕ К ДОКУМЕНТУ

Static-подготовка к dynamic function tables

Этот раздел добавлен как развитие проекта после уточнения важной архитектурной границы: .pdata в PE-файле не является всей вселенной unwind-информации. Static-слой видит файл на диске и не может увидеть runtime table, которая еще не создана. Но static может собрать признаки подготовки к такой таблице и сформулировать гипотезу, которую должен проверить dynamic-слой.

Текущая версия проекта в основном анализирует уже объявленную PE EH/Unwind-карту: Exception Directory, RUNTIME_FUNCTION, UNWIND_INFO, handler placement, reuse/dominance, structural integrity. Она не выполняет полноценный import analysis, не ищет executable memory allocation patterns и не сканирует данные на runtime RUNTIME_FUNCTION-like structures. Поэтому описанное ниже является не исправлением текущего бага, а естественным направлением развития static-слоя.

10.1. Зачем static-слою искать подготовку к runtime tables

Dynamic function table появляется только во время выполнения. Это значит, что static не может прочитать ее как готовую таблицу из PE Exception Directory. Однако подготовка к ней часто оставляет следы в файле: imports, строки имен API, call patterns, wrapper-функции, выделение executable memory, массивы структур, похожих на RUNTIME_FUNCTION, callback-код для поиска записи по ControlPc.

Правильная цель static-слоя здесь не финальный вердикт, а гипотеза. Static должен сказать: "в файле есть признаки того, что target может генерировать или обслуживать код во время выполнения и регистрировать для него dynamic unwind metadata". После этого dynamic должен проверить, действительно ли при запуске вызываются runtime table APIs и появляется ли execution внутри зарегистрированного диапазона.

10.2. Признак 1: imports и resolved API names

Первый и самый дешевый признак - Import Directory. Если PE импортирует функции, связанные с dynamic function tables, это прямое evidence-наблюдение: программа потенциально работает с runtime unwind map. Но import не равен доказательству использования. Функция может быть импортирована библиотекой "на всякий случай", wrapper-слоем, профилировщиком, языковым runtime-ом или нормальным JIT engine.

Минимальный список API, который стоит добавить в static checks: RtlAddFunctionTable, RtlInstallFunctionTableCallback, RtlDeleteFunctionTable, RtlAddGrowableFunctionTable, RtlGrowFunctionTable, RtlDeleteGrowableFunctionTable. Дополнительно полезно искать GetProcAddress и LoadLibraryA/W, потому что отсутствие прямого import не доказывает отсутствие механизма: API может резолвиться динамически по имени, ordinal, hash или через собственный resolver.

Группа API	Что означает как static evidence
RtlAddFunctionTable	Программа может регистрировать массив RUNTIME_FUNCTION во время выполнения. Это главный прямой сигнал runtime unwind table.
RtlInstallFunctionTableCallback	Программа может регистрировать callback-based function table, где записи предоставляются по запросу. Это особенно важно для JIT/runtime-generated code.
RtlDeleteFunctionTable	Программа может удалять ранее зарегистрированную dynamic table; это указывает на жизненный цикл runtime unwind map.
RtlAddGrowableFunctionTable / RtlGrowFunctionTable / RtlDeleteGrowableFunctionTable	Программа может вести изменяемую runtime table, которая растет или меняется после запуска. Это расширяет покрытие JIT/runtime сценариев.
GetProcAddress / LoadLibrary	Не доказывают use of runtime tables, но объясняют, почему прямой import может отсутствовать. Требуют поиска строк имен API, hash-resolver patterns или thunk wrappers.

В отчете это должно выглядеть не как "обнаружен JIT", а как "static evidence: import/use of dynamic function table API". Severity такого признака должна быть слабой или информационной без дополнительных подтверждений, потому что benign explanations очень вероятны.

10.3. Признак 2: executable memory lifecycle

Второй признак - жизненный цикл исполняемой памяти. Если программа выделяет память, записывает туда bytes, меняет protection на executable и затем передает управление в эту область, static должен спросить: есть ли рядом регистрация function table? Для Windows x64 это важно, потому что nonleaf runtime-generated code без корректной unwind metadata может ломать stack walking, exception dispatch и crash analysis.

Для реализации нужна не только таблица imports, но и lightweight code-pattern layer. Он должен искать вызовы VirtualAlloc, VirtualAlloc2, HeapAlloc, NtAllocateVirtualMemory, MapViewOfFile, VirtualProtect, NtProtectVirtualMemory, WriteProcessMemory, memcpy-like writes, а затем переходы call/jmp в область, полученную из выделения. Без дизассемблирования это можно сделать лишь частично через imports/strings; полноценная версия требует disassembly и data-flow хотя бы на уровне basic blocks.

Важно: executable memory само по себе не является аномалией. JIT, scripting engines, profilers, emulators, packers, protectors и legitimate runtimes могут использовать ее нормально. Сильным становится не один признак, а связка: executable memory lifecycle плюс dynamic function table API плюс runtime hit или exception внутри зарегистрированного диапазона.

Наблюдение	Как трактовать
VirtualAlloc/NtAllocateVirtualMemory + PAGE_EXECUTE*	Возможное создание исполняемой памяти. Требует проверки, будет ли для этой области unwind registration.
VirtualProtect/NtProtectVirtualMemory переводит RW -> RX/RWX	Похоже на подготовку generated code region или unpacked code. Не является verdict без контекста.
Запись bytes перед исполнением	Усиливает гипотезу runtime-generated code, особенно если затем есть indirect call/jmp.
Переход в выделенную область	Сильный runtime/control-flow сигнал, но static его надежно доказывает только при наличии disassembly/data-flow.
Рядом есть RtlAddFunctionTable или callback registration	Формирует гипотезу: generated code может иметь dynamic unwind metadata. Dynamic должен подтвердить actual registration.

10.4. Признак 3: внутренние структуры runtime-а

Третий признак глубже imports. Иногда в бинаре можно найти структуры, похожие на будущие RUNTIME_FUNCTION entries: последовательности begin/end/unwind-like полей, сортированные диапазоны, таблицы указателей на xdata-like blobs, callback-функцию, которая по ControlPc ищет подходящую запись. Это уже evidence chain, а не одиночный import.

Такой поиск сложнее и рискованнее по false positives. Любые три 32-bit числа могут случайно напоминать RUNTIME_FUNCTION. Поэтому static scanner должен применять строгие фильтры: Begin < End, диапазоны внутри одного plausible generated-code base, выравнивание, монотонная сортировка, UnwindInfo-like header по адресу, допустимые Version/Flags/CountOfCodes, отсутствие выхода за mapped data, связка с вызовом RtlAddFunctionTable или callback registration.

Этот модуль не должен сразу повышать severity до high. Лучше вводить отдельный тип evidence: possible_runtime_function_table_construction. Он должен давать analyst hint и manual validation steps: проверить data block, xrefs, call site, base address, handler-related code и dynamic confirmation.

Паттерн	Фильтр против ложных срабатываний
Тройки u32, похожие на Begin/End/UnwindInfo	Begin < End, разумный размер функции, много записей подряд, сортировка по Begin.
UnwindInfo-like blobs	Version 1/2, корректные flags, CountOfCodes не выходит за blob, tail читается по правилам padding.
Callback-функция поиска ControlPc	Наличие сравнений диапазонов, binary search/linear search по таблице, возврат указателя на entry.
Xrefs к RtlAddFunctionTable	Передача pointer/count/base в регистры RCX/RDX/R8 перед вызовом.
Связь с executable memory	BaseAddress связан с allocated/generated region, а не с обычной .text статического image.

10.5. Новая static-гипотеза в отчете

В отчет нужно добавить отдельный блок не как verdict, а как hypothesis. Он должен показывать, какие static evidence были найдены, почему они важны и что именно dynamic должен подтвердить. Такой стиль дисциплинирует анализ: static не притворяется, что знает runtime-факт без запуска, а честно передает dynamic-слою проверяемую гипотезу.

```
Hypothesis: target may generate code at runtime and register dynamic unwind metadata
Static evidence:
  - import/use of RtlAddFunctionTable or callback/growable table API
  - executable memory allocation or protection change to executable
  - memory writes before possible execution
  - possible RUNTIME_FUNCTION array construction
  - possible callback that maps ControlPc to function entries
Needed dynamic validation:
  - confirm actual API call
  - capture FunctionTable / EntryCount / BaseAddress or Callback / Length
  - snapshot runtime table from target memory
  - observe execution inside registered range
  - correlate exception address, runtime entry and handler route
```

Такой блок особенно полезен на защите: он показывает, что проект мыслит static и dynamic как две части доказательной системы, а не как две несвязанные кнопки.

10.6. Как внедрить без поломки текущего ядра

Внедрять это нужно пошагово, без резкой перестройки ядра. Первый этап - расширить PE parser: добавить Import Directory parser, список imported DLL/function names и JSON/HTML output для imports of interest. Это почти не трогает unwind.c и минимально рискует сломать текущую .pdata/.xdata логику.

Второй этап - добавить новый evidence family, например runtime_table_preparation. Он должен жить рядом с heuristics, но не смешиваться с table_integrity/unwind_integrity. Его признаки должны быть weak/informational,

пока нет подтверждения dynamic. Это важно, чтобы проект не начал шуметь на нормальных JIT/runtime программах.

Третий этап - добавить static scanner для strings/GetProcAddress. Он должен искать имена runtime table API в строках и рядом фиксировать use of GetProcAddress/LoadLibrary. Это покрывает случаи, когда прямого import нет, но API резолвится динамически.

Четвертый этап - добавить lightweight disassembly layer. Сначала только для call sites вокруг интересующих imports: какие регистры готовятся перед вызовом RtlAddFunctionTable, есть ли pointer/count/base, есть ли nearby VirtualAlloc/VirtualProtect. Полный data-flow можно оставить на будущее.

Пятый этап - добавить runtime table snapshot в dynamic: когда dynamic поймал RtlAddFunctionTable, он должен через ReadProcessMemory прочитать массив RUNTIME_FUNCTION из target memory, проверить entries и добавить их в отдельный report block. Это будет первое сильное соединение новой static-гипотезы и dynamic validation.

10.7. Приоритеты реализации

Приоритет	Улучшение	Почему именно так
P1	Import Directory parser и список dynamic function table APIs	Самое безопасное и быстрое расширение static. Позволит ловить прямые признаки подготовки без дизассемблера.
P1	HTML/JSON блок Dynamic table preparation hypothesis	Сразу улучшает отчетность и защиту проекта, даже если признаки пока weak/informational.
P2	GetProcAddress/LoadLibrary/string scanner	Закрывает частый обход direct imports. Реализуется проще, чем полноценный disassembly.
P2	Поддержка growable function table APIs в dynamic breakpoints	Расширяет runtime coverage: RtlAddGrowableFunctionTable/RtlGrowFunctionTable/RtlDeleteGrowableFunctionTable.
P3	Static executable memory pattern layer	Нужен disassembly/call-site analysis; полезно, но требует аккуратных тестов против false positives.
P3	Runtime table snapshot через ReadProcessMemory	Очень важное улучшение dynamic: не только факт вызова API, но и содержимое таблицы.
P4	Поиск RUNTIME_FUNCTION-like arrays в data sections	Мощно, но риск false positives выше; делать после появления строгих фильтров и тестового корпуса.
P4	Per-function static/dynamic proof graph	Идеальный результат: static hypothesis -> runtime registration -> execution inside range -> exception/handler route.

10.8. Какие тестовые бинарники нужны для этой доработки

Для демонстрации новой static-гипотезы нужен отдельный набор controlled binaries. Их задача - не просто сломать .xdata, а показать все комбинации static evidence и dynamic validation.

Тест	Что содержит	Ожидаемый вывод
direct_import_rtl_add_function_table	Прямой import RtlAddFunctionTable, но API не вызывается.	Static hypothesis weak: import evidence есть, dynamic confirmation нет.
direct_import_and_call_rtl_add_function_table	Import и реальный вызов RtlAddFunctionTable.	Static hypothesis + dynamic event rtl_add_function_table.
getprocaddress_rtl_add_function_table	Нет прямого import, имя API берется через GetProcAddress.	Static string/resolver evidence; dynamic подтверждает при вызове.
growable_function_table	Вызов RtlAddGrowableFunctionTable/RtlGrowFunctionTable.	Проверка расширенного dynamic coverage.
exec_memory_no_table	VirtualAlloc/VirtualProtect RX и исполнение, но без function table registration.	Static warning: generated code without observed unwind registration; dynamic должен подтвердить/опровергнуть.
exec_memory_with_table	Generated code region плюс RtlAddFunctionTable.	Сильная цепочка: executable memory + runtime registration.
runtime_function_like_false_positive	Data похожи на triples u32, но не проходят строгие фильтры.	Проверка, что static scanner не шумит.
callback_table_registration	RtlInstallFunctionTableCallback и callback address.	Dynamic event callback table; future static должен найти callback-like search routine.

10.9. Формулировка для защиты

Если комиссия спросит, почему static не показывает runtime function table, ответ должен быть строгим: static работает с PE-файлом на диске, а runtime table создается в процессе после запуска. Поэтому static не может прочитать уже созданную runtime table заранее. Но развитый static-слой может обнаружить подготовку: imports, dynamic API resolution, executable memory lifecycle и структуры, похожие на RUNTIME_FUNCTION arrays.

Текущая версия проекта уже имеет dynamic-проверку для RtlAddFunctionTable и RtlInstallFunctionTableCallback, но static-гипотезу подготовки пока не строит. Это честно оформляется как направление развития: добавить import analysis, scanner for runtime table APIs, executable-memory pattern layer, possible runtime table construction detector и затем связать это с dynamic snapshot.

Главная фраза: хороший анализатор не должен делать вид, что знает runtime-факты без runtime-а. Он должен разделять static evidence и dynamic validation. Static формулирует гипотезу; dynamic подтверждает или не подтверждает ее в конкретной execution session.